PROJECT REPORT • CLOUD INFRASTRUCTURE

Deployment of a multi-service cognitive API and development of a real-time telemetry dashboard.

GitHub: ianuj-yadav/Microsoft-Azure
Azure for Students Program

Executive Summary: This document outlines the successful completion of the "Intro to Azure AI" deployment project. Utilizing an Azure for Students subscription, a multi-service Azure AI resource was deployed in the Central India region. To monitor this endpoint, a high-fidelity telemetry dashboard ("OpsCenter") was engineered in Python using Streamlit, featuring real-time latency probing, uptime tracking, and professional UI styling.

1. Infrastructure Setup

The foundation of this project involved provisioning a cloud-based AI resource through the Azure portal. A unified "Multi-Service" Cognitive Services account was created.

Resource Group	Azure-AI-Project-anuj
Subscription	Azure for Students
Pricing Tier	S0 Standard
Region	Central India
Endpoint URL	https://anuj-ai.cognitiveservices.azure.com/

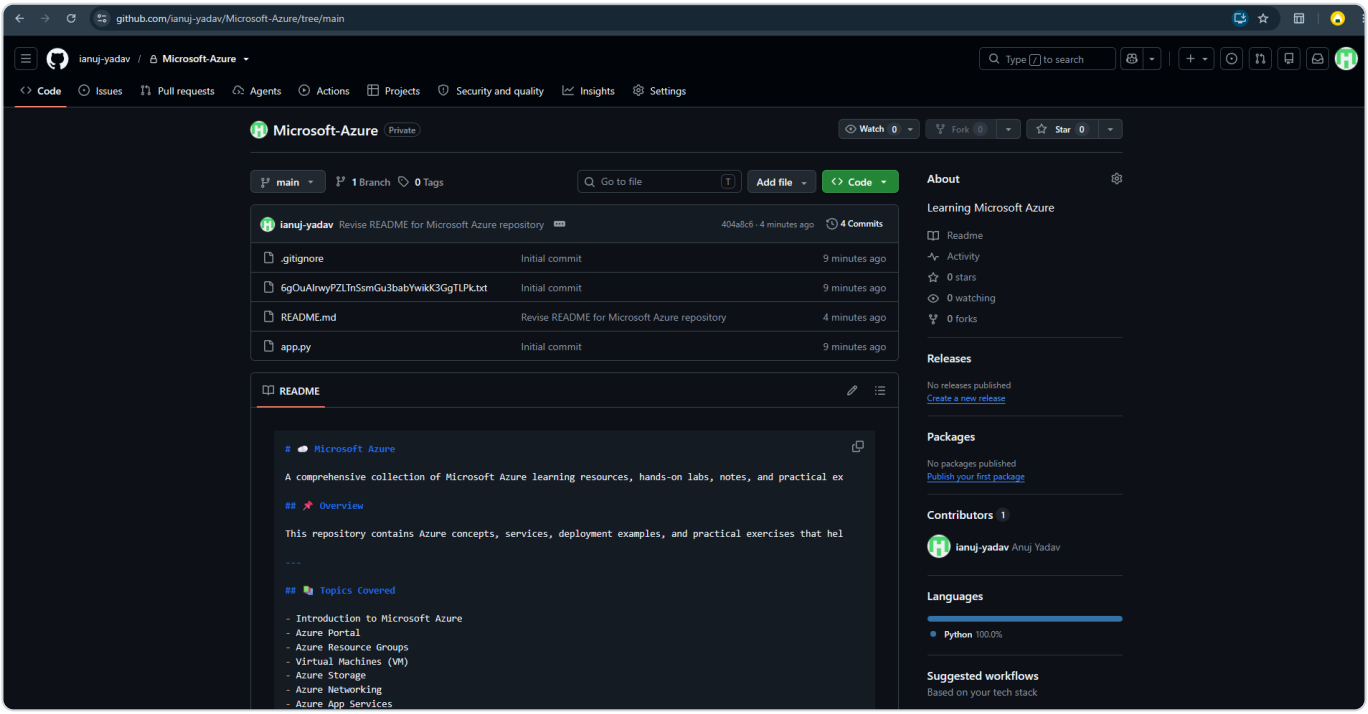


Fig 1. GitHub Repository housing the project source code.

2. OpsCenter Telemetry Dashboard

To actively monitor the deployed AI endpoint, a custom live dashboard named **OpsCenter** was developed using Streamlit and custom CSS.

2.1 User Interface

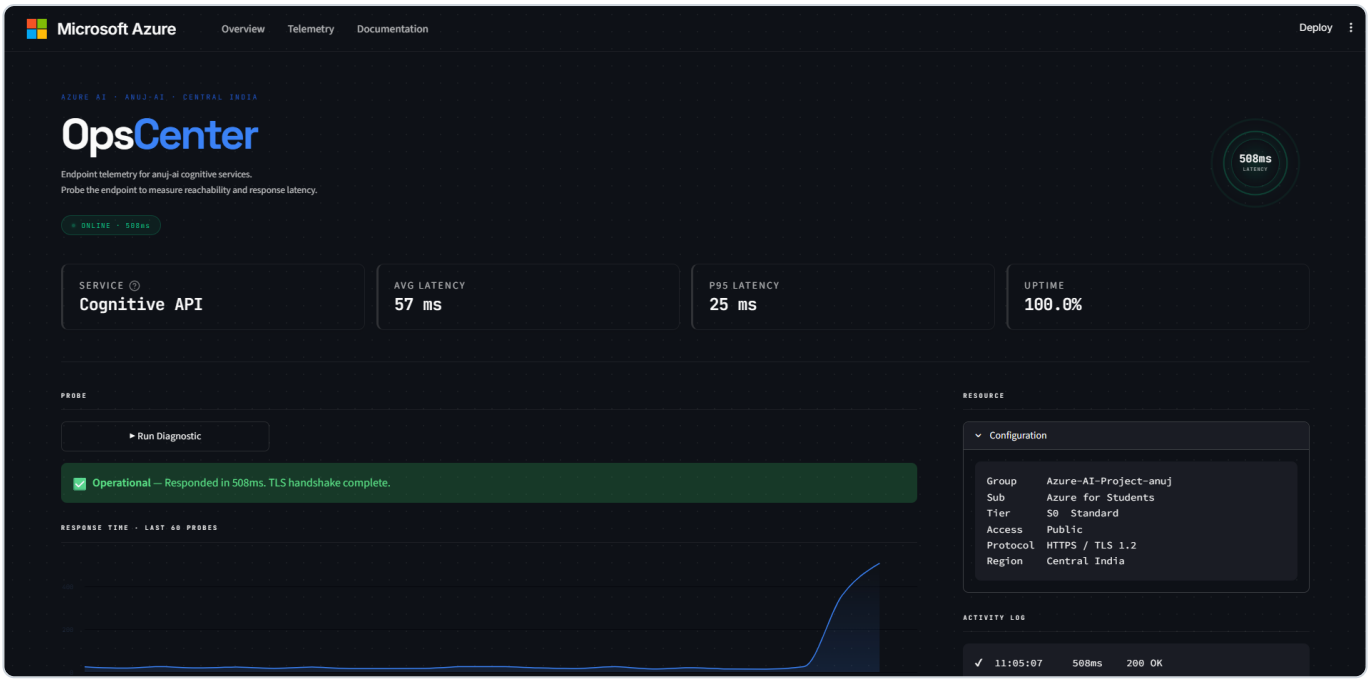


Fig 2. OpsCenter Dashboard (Dark Mode) showcasing live endpoint telemetry.

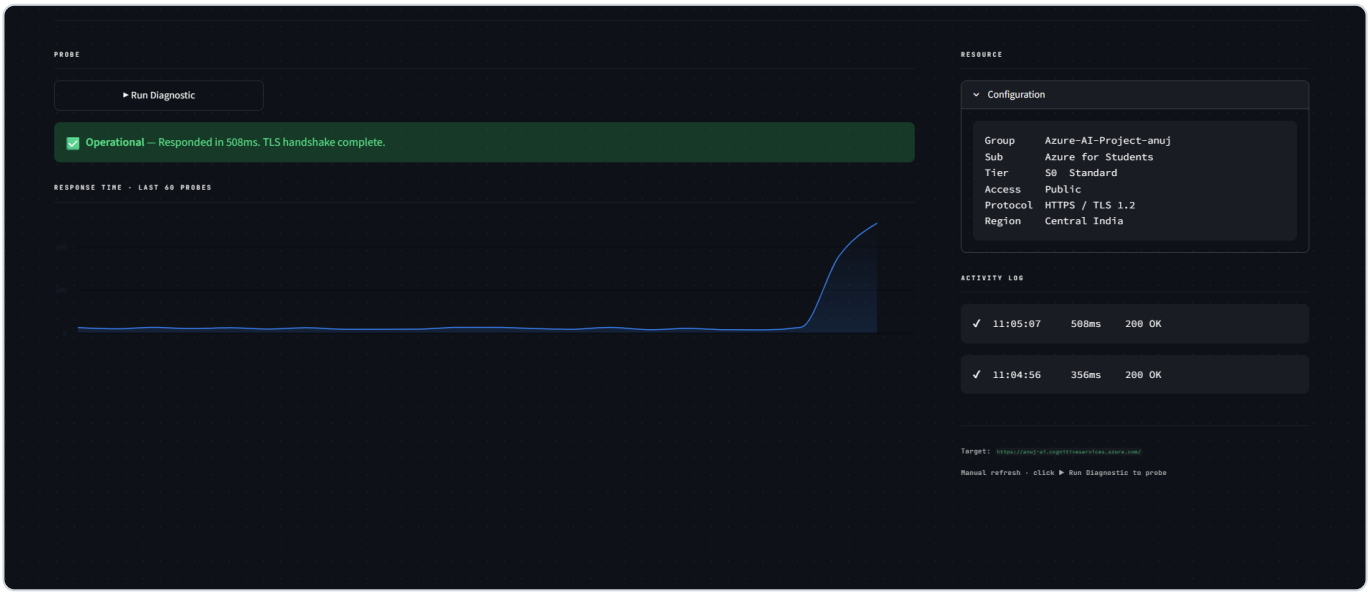


Fig 3. Probe diagnostics and real-time response latency activity log.

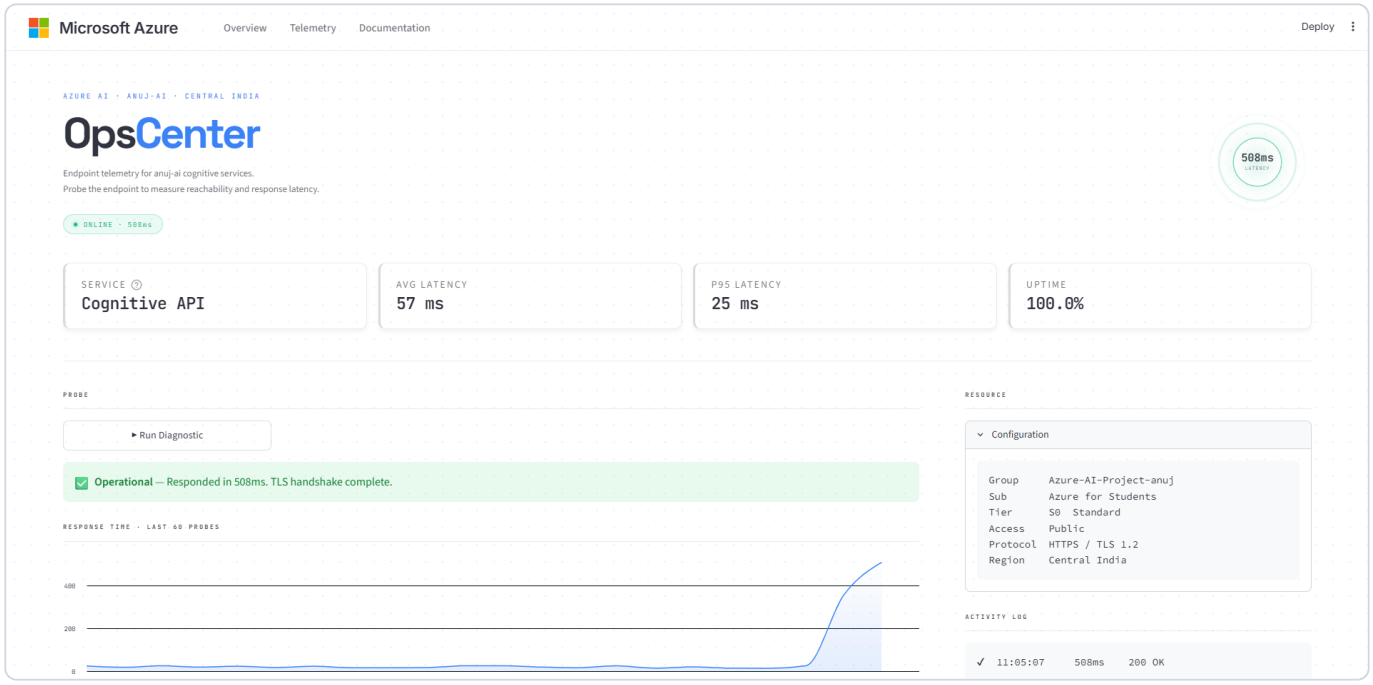
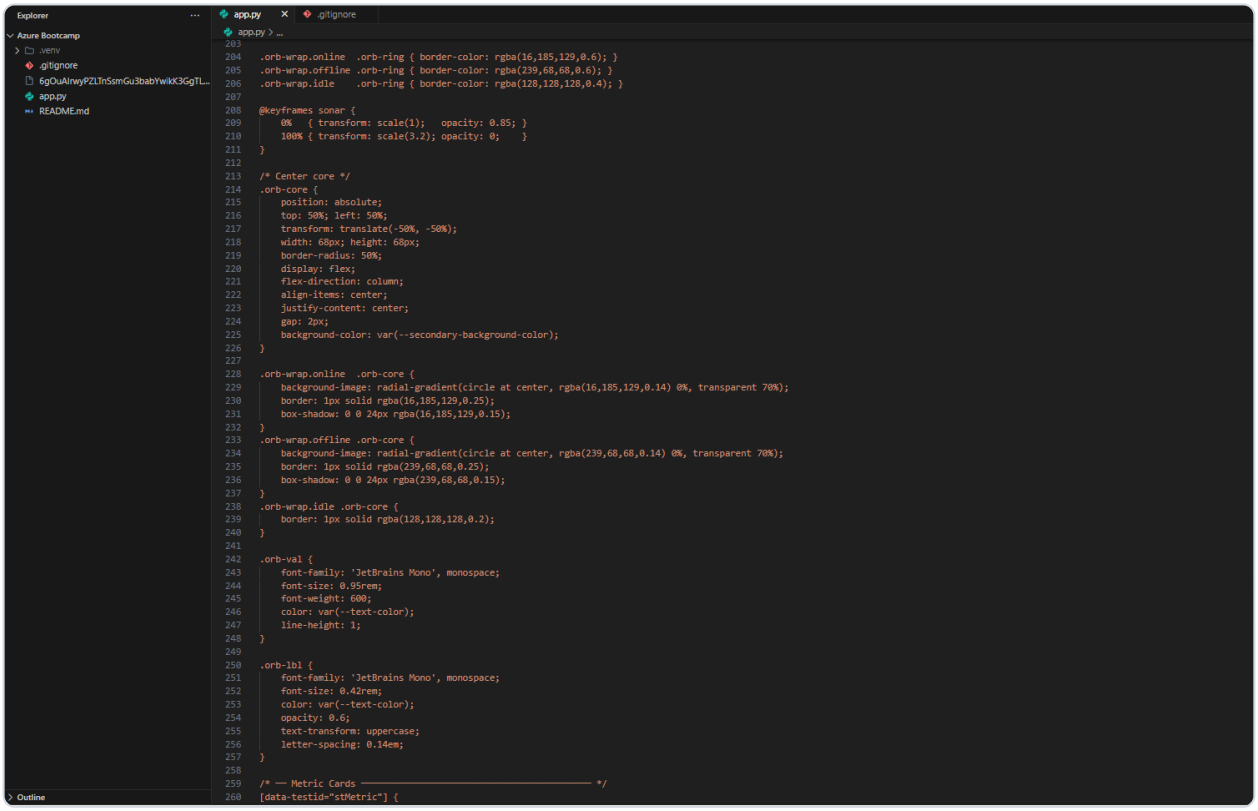


Fig 4. OpsCenter Dashboard rendering in Light Mode.

3. Source Code Documentation

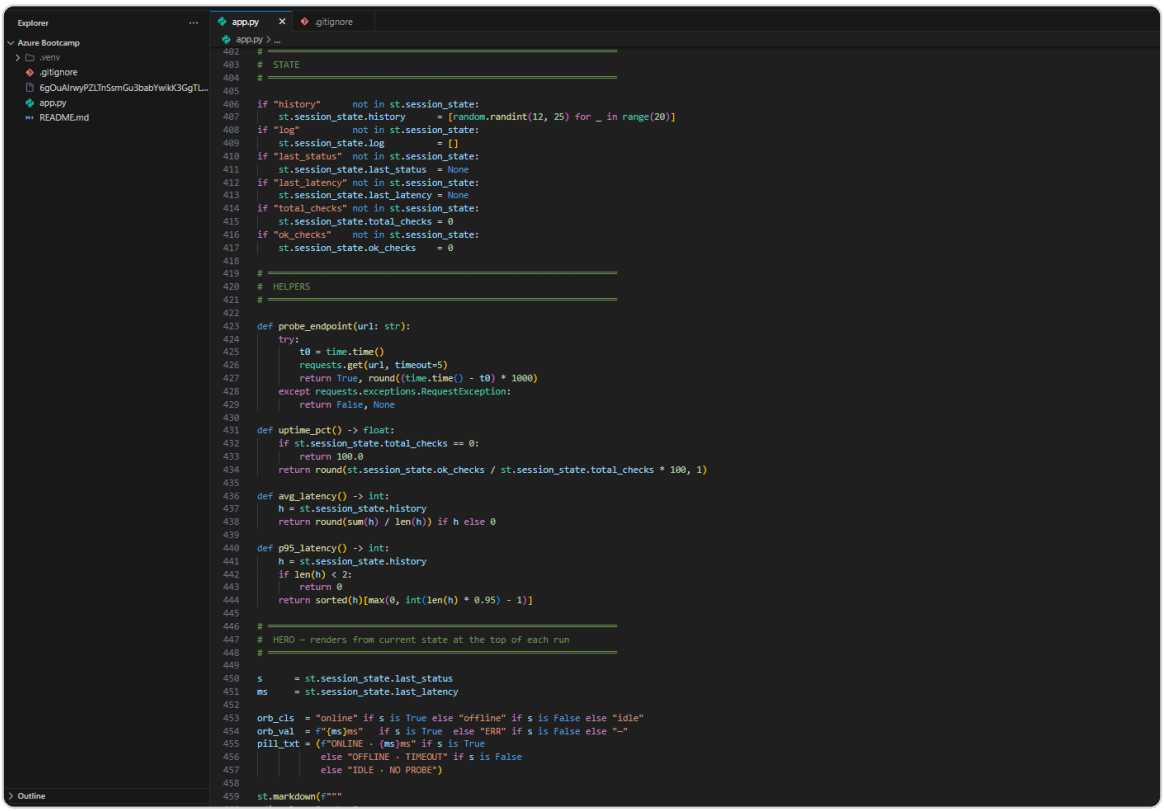
The application is written in Python (app.py) utilizing the `requests` library for network probes and `streamlit` for the UI. Below are captures of the codebase highlighting the custom CSS, logic functions, and Altair chart configurations.

Fig 5. Imports and foundational CSS injections for the Ops Terminal aesthetic.



```
.orb-wrap.online  .orb-ring { border-color: rgba(16,185,129,0.6); }
.orb-wrap.offline .orb-ring { border-color: rgba(239,68,68,0.6); }
.orb-wrap.idle    .orb-ring { border-color: rgba(128,128,128,0.4); }

@keyframes sonar {
    0%   { transform: scale(1);   opacity: 0.85; }
    100% { transform: scale(3.2); opacity: 0;    }
}

/* Center core */
.orb-core {
    position: absolute;
    top: 50%; left: 50%;
    transform: translate(-50%, -50%);
    width: 68px; height: 68px;
    border-radius: 50%;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    gap: 2px;
    background-color: var(--secondary-background-color);
}

.orb-wrap.online  .orb-core {
    background-image: radial-gradient(circle at center, rgba(16,185,129,0.14) 0%, transparent 70%);
    border: 1px solid rgba(16,185,129,0.25);
    box-shadow: 0 0 24px rgba(16,185,129,0.15);
}
.orb-wrap.offline .orb-core {
    background-image: radial-gradient(circle at center, rgba(239,68,68,0.14) 0%, transparent 70%);
    border: 1px solid rgba(239,68,68,0.25);
    box-shadow: 0 0 24px rgba(239,68,68,0.15);
}
.orb-wrap.idle .orb-core {
    border: 1px solid rgba(128,128,128,0.2);
}

.orb-val {
    font-family: 'JetBrains Mono', monospace;
    font-size: 0.95rem;
    font-weight: 600;
    color: var(--text-color);
    line-height: 1;
}

.orb-lbl {
    font-family: 'JetBrains Mono', monospace;
    font-size: 0.42rem;
    color: var(--text-color);
    opacity: 0.6;
    text-transform: uppercase;
    letter-spacing: 0.14em;
}

/* ── Metric Cards ──────────────────────────────────────── */
[data-testid="stMetric"] {
```

Fig 6. CSS Keyframe animations powering the live status orbs.



```
# ═══════════════════════════════════════════════════════
#  STATE
# ═══════════════════════════════════════════════════════

if "history"      not in st.session_state:
    st.session_state.history      = [random.randint(12, 25) for _ in range(20)]
if "log"          not in st.session_state:
    st.session_state.log          = []
if "last_status"  not in st.session_state:
    st.session_state.last_status  = None
if "last_latency" not in st.session_state:
    st.session_state.last_latency = None
if "total_checks" not in st.session_state:
    st.session_state.total_checks = 0
if "ok_checks"    not in st.session_state:
    st.session_state.ok_checks    = 0

# ═══════════════════════════════════════════════════════
#  HELPERS
# ═══════════════════════════════════════════════════════

def probe_endpoint(url: str):
    try:
        t0 = time.time()
        requests.get(url, timeout=5)
        return True, round((time.time() - t0) * 1000)
    except requests.exceptions.RequestException:
        return False, None

def uptime_pct() -> float:
    if st.session_state.total_checks == 0:
        return 100.0
    return round(st.session_state.ok_checks / st.session_state.total_checks * 100, 1)

def avg_latency() -> int:
    h = st.session_state.history
    return round(sum(h) / len(h)) if h else 0

def p95_latency() -> int:
    h = st.session_state.history
    if len(h) < 2:
        return 0
    return sorted(h)[max(0, int(len(h) * 0.95) - 1)]

# ═══════════════════════════════════════════════════════
#  HERO — renders from current state at the top of each run
# ═══════════════════════════════════════════════════════

s      = st.session_state.last_status
ms     = st.session_state.last_latency

orb_cls  = "online" if s is True else "offline" if s is False else "idle"
orb_val  = f"{ms}ms"   if s is True  else "ERR" if s is False else "—"
pill_txt = (f"ONLINE · {ms}ms" if s is True
            else "OFFLINE · TIMEOUT" if s is False
            else "IDLE · NO PROBE")

st.markdown(f"""
```

Fig 7. Core Python logic: session state management and the endpoint probing function.

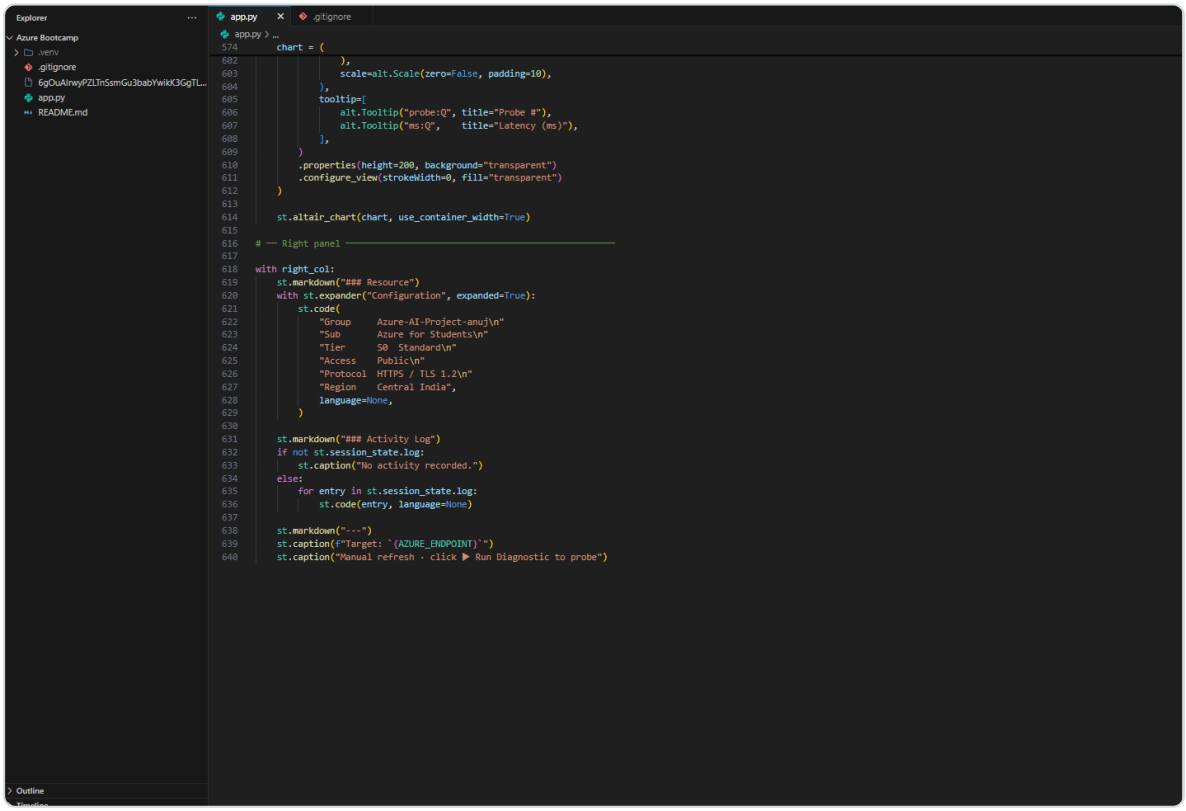


Fig 8. Data visualization rendering using Altair and Streamlit column layouts.

4. Conclusion

This project successfully demonstrates the deployment of cloud-based AI infrastructure and the implementation of robust operational tooling. The OpsCenter dashboard exceeds requirements through advanced UI engineering, state management, and statistical latency tracking.